encamina

PIENSA EN COLORES

Una consultora tecnológica que piensa en colores

Microsoft 2025 Partner of the Year Winner Spain

Microsoft Solutions Partner
Digital & App Innovation
Azure

Microsoft Solutions Partner
Data & AI
Azure

Microsoft Solutions Partner
Infrastructure
Azure

Microsoft Solutions Partner
Modern Work

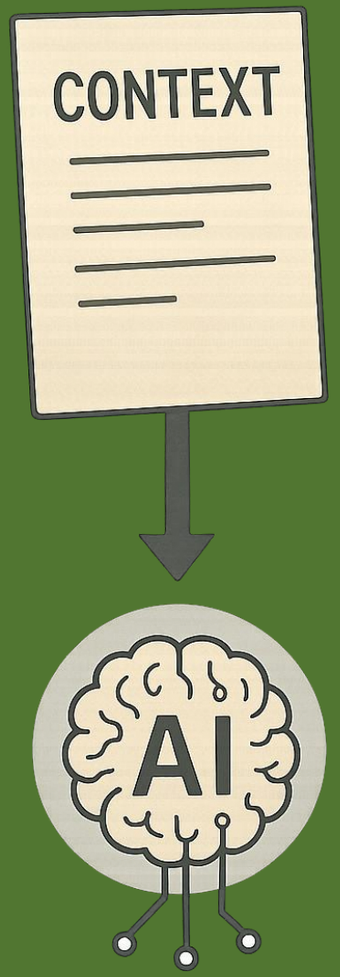
Microsoft Solutions Partner
Business Applications

Microsoft Solutions Partner
Security



Trabajando con GitHub Spec Kit

Openfinance



Context Coding

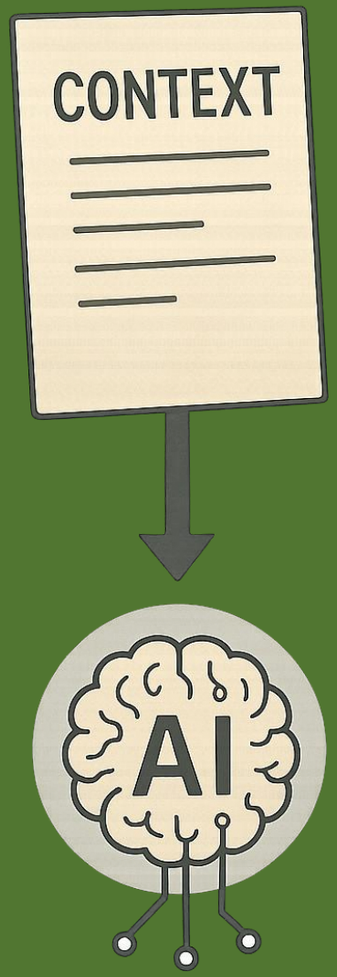
Herramientas como GitHub Copilot están impulsando un cambio sistémico potenciado por modelos de IA generativa cada vez más sofisticados.

Estas herramientas buscan incrementar la productividad, automatizando tareas mundanas e incluso generando documentación y lógica compleja.

Sin embargo, muchas personas, sobre todo desarrolladores, se encuentran oscilando entre momentos de asombro y frustrantes ciclos de ajuste de *prompts*.

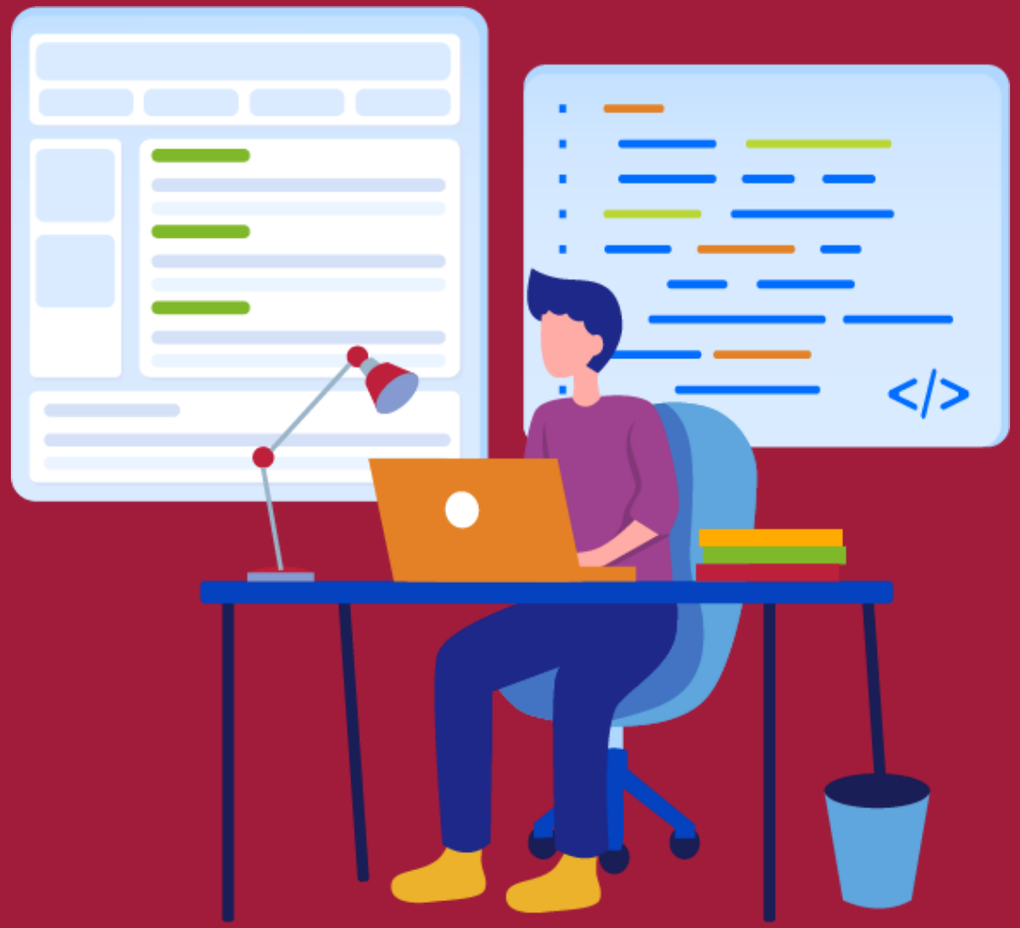
¿Por qué?

Context Coding



La respuesta realmente no reside en las limitaciones de la tecnología, sino en los humanos…

En la era de la IA,
la habilidad más crítica no es (solo) escribir código,
sino crear contexto.



Spec-Driven Development
SDD

Es un enfoque de desarrollo **donde las especificaciones estructuradas** (*specs*) son el elemento principal que guía a agentes de IA para generar artefactos (código o documentación) de forma consistente y con menos alucinaciones, con el componente humano en el centro, siempre como último y único responsable.

Así como un arquitecto civil necesita planos antes de construir, un agente de IA necesita *specs* antes de implementar.

SDD – Spec-Driven Development

SIN SDD	CON SDD
En proyectos compartidos o gestionados por un equipo, la IA genera código inconsistente e impredecible	Las *specs* aportan contexto estructurado y psuedo-determinista que se preserva como parte del código que comparte cada persona dentro del equipo
Alucinaciones frecuentes en las salidas	Con una aproximación de *«Human-in-the-Loop»* se valida cada etapa critica, minimizando el impacto negativo de las alucinaciones
Contexto perdido entre sesiones	Se preserva el contexto entre iteraciones.
Retrabajo constante y alto costo, elevada frustración	Flujo relativamente predecible y reproducible, y siempre auditable
Documentación desactualizada o ausente	Las *specs* son documentación viva y versionada, son parte constante del trabajo entre el humano con la IA
Onboarding lento e inconsistente	Las *specs* aceleran el *onboarding* y la transferencia de conocimientos

SDD – Spec-Driven Development



Predictibilidad

La IA sigua una especificación, y reducir la desviación de requisitos.



Menos Alucinaciones

El contexto estructurado reduce drásticamente las alucinaciones del modelo (no las elimina).



Documentación Viva

La documentación evoluciona junto con el proyecto, y refleja el estado actual del producto.



Trazabilidad

Cada decisión técnica se vincula a una especificación aprobada.



Velocidad

Menos ciclos de revisión implican una entrega más rápida y un menor coste de cambio.



Comunicación

Las especificaciones alinean a desarrolladores y *stakeholders* alrededor de una única fuente de verdad.

encamina

SDD – Spec-Driven Development

Es una propuesta para evolucionar en la era de la IA



Spec-First

La IA sigue la especificación y reduce la desviación de requisitos. Las *specs* se escriben siempre antes que cualquier código.



Architecture-First

El contexto estructurado reduce drásticamente las alucinaciones del modelo. Ejemplos: PRDs, ADRs, instructions, y skills…



Human-in-the-Loop

Cada decisión técnica se vincula a una especificación aprobada. La validación humana siempre en todas y cada una de las etapas.



Rastreabilidad

Menos ciclos de revisión implican una entrega más rápida y un menor coste de cambio. Cada decisión siempre viene de una *spec* aprobada por el ser humano.



No Improvisación

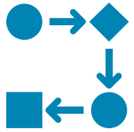
Reduce la probabilidad de que la IA improvise, siempre y cuando se proporcione una especificación bien redactada y concebida.

SDD – Spec-Driven Development

Reduce la ambigüedad, **pero** no reemplaza:



Las pruebas automatizadas



Revisiones del código (siempre por un humano, opcional por otra IA)



Revisiones y auditorias de seguridad (*threat modeling*) y cumplimiento



Las evaluaciones, sean cualitativas, orgánicas o sintéticas

SDD – Spec-Driven Development

BMAD-METHOD

PromptX

SPEC KIT

OpenSQZ/
GTPlanner

GTPlanner: AI-Powered PRD Generation Tool

OPENSPEC

Builder Methods
Agent OS

encamina





GitHub Spec Kit

Es un conjunto de herramientas de código abierto mantenido por GitHub que ayuda a los equipos a centrarse en los escenarios de producto y en resultados predecibles, en lugar de hacer «*vibe coding*» para cada artefacto desde cero.

Operacionaliza el SDD con una CLI y comandos *slash* a modo de agentes, empleando un asistente de codificación como plataforma, y un flujo o etapas definidas de realización.

GitHub Spec Kit

- **Open-Source**, con licencia MIT
- **+140k estrellas**, siendo un repositorio muy popular
- **+310 colaboradores <u>humanos</u>** directos y constantes
- **Gestionado** directamente **por GitHub**
- **Fácil de usar**, basado en un CLI y luego en *skills* invocables como comandos *slash*.

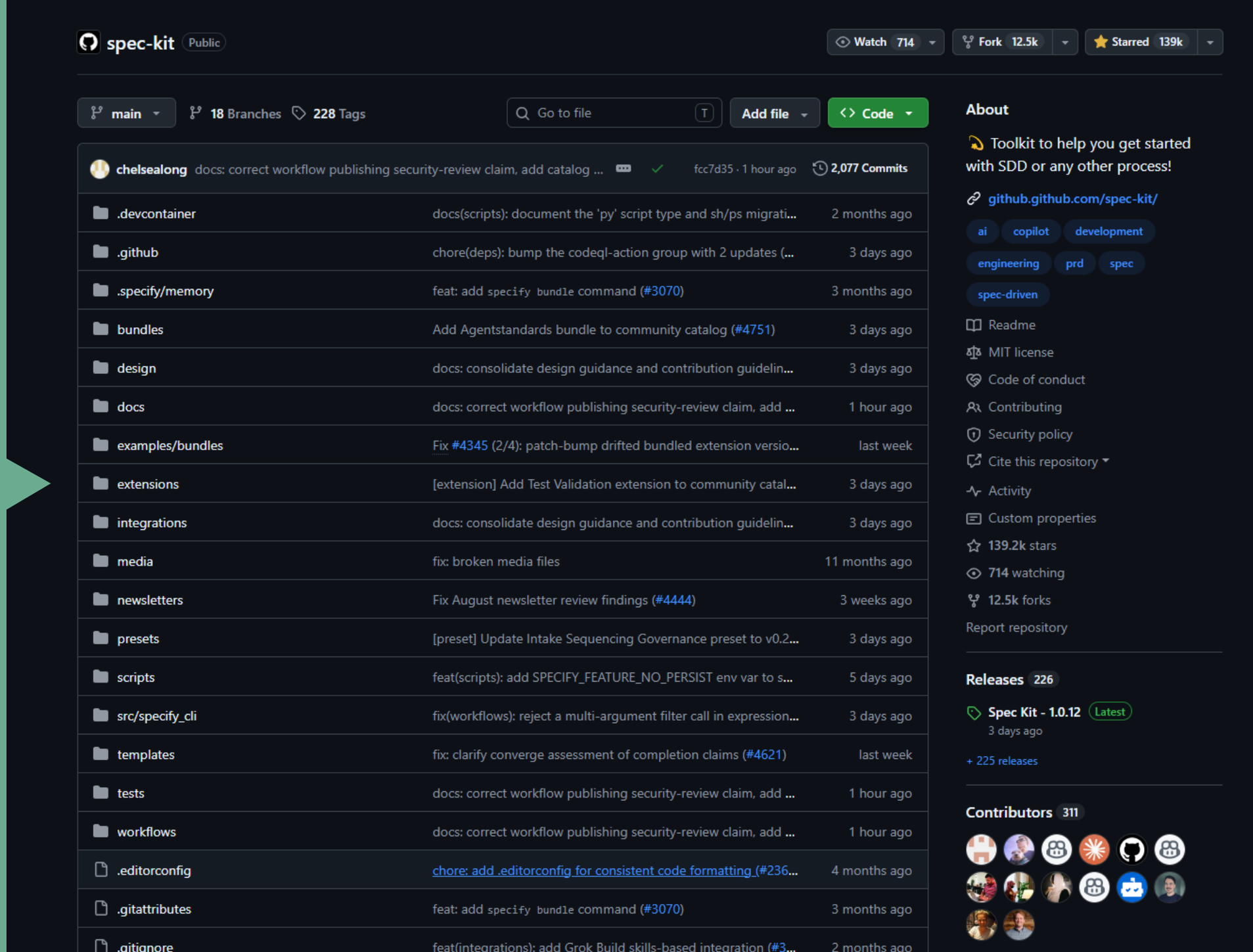


Inspirado en la investigación de John Lam, sobre cómo ayudar a dirigir el proceso de desarrollo de software con grandes modelos de lenguaje (LLMs), haciéndolos un poco más deterministas

GitHub Spec Kit

Y muchísimos más, y con cada iteración se agregan nuevos…

GitHub Spec Kit



GitHub Spec Kit

Instalación

Opción 1 — Uso puntual (sin instalación)

```
uvx --from git+https://github.com/github/spec-kit.git specify init <PROJECT_NAME>
```

Opción 2 — Instalación permanente (recomendada)

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
```

📋 **Requisitos previos:** Python 3.11+, `uv cli`, `git cli`, y un asistente de codificación (en nuestro caso GitHub Copilot)

encamina

GitHub Spec Kit

Inicialización en un proyecto

```
# Nuevo proyecto con GitHub Copilot
specify init my-project --ai copilot

# En un proyecto existente
specify init --here --ai copilot

# Windows (PowerShell)
specify init my-project --ai copilot --script ps

# Con AI Skills integradas
specify init my-project --ai copilot --ai-skills

# Con Claude Code
specify init my-project --ai claude
```

🆙 **Para actualizar:** simplemente volver a ejecutar el comando `specify init`

GitHub Spec Kit

Comandos Slash /

Comandos esenciales – CORE

COMANDO	FUNCIÓN O PROPÓSITO
`/speckit.constitution`	Crea los principios, guías, normas y estatutos que regirán la vida del proyecto.
`/speckit.specify`	Define qué construir. Las especificaciones en forma de requisitos (*features* + *user stories*).
`/speckit.plan`	Genera el plan técnico de implementación. Aquí van los detalles de implementación e información adicional del *tech stack*, arquitectura y limitaciones técnicas.
`/speckit.tasks`	Crea una lista de tareas accionables y priorizadas sobre el plan técnico.
`/speckit.implement`	Construye e implementa efectivamente el proyecto (codifica), ejecutando las tareas siguiendo el plan.

GitHub Spec Kit

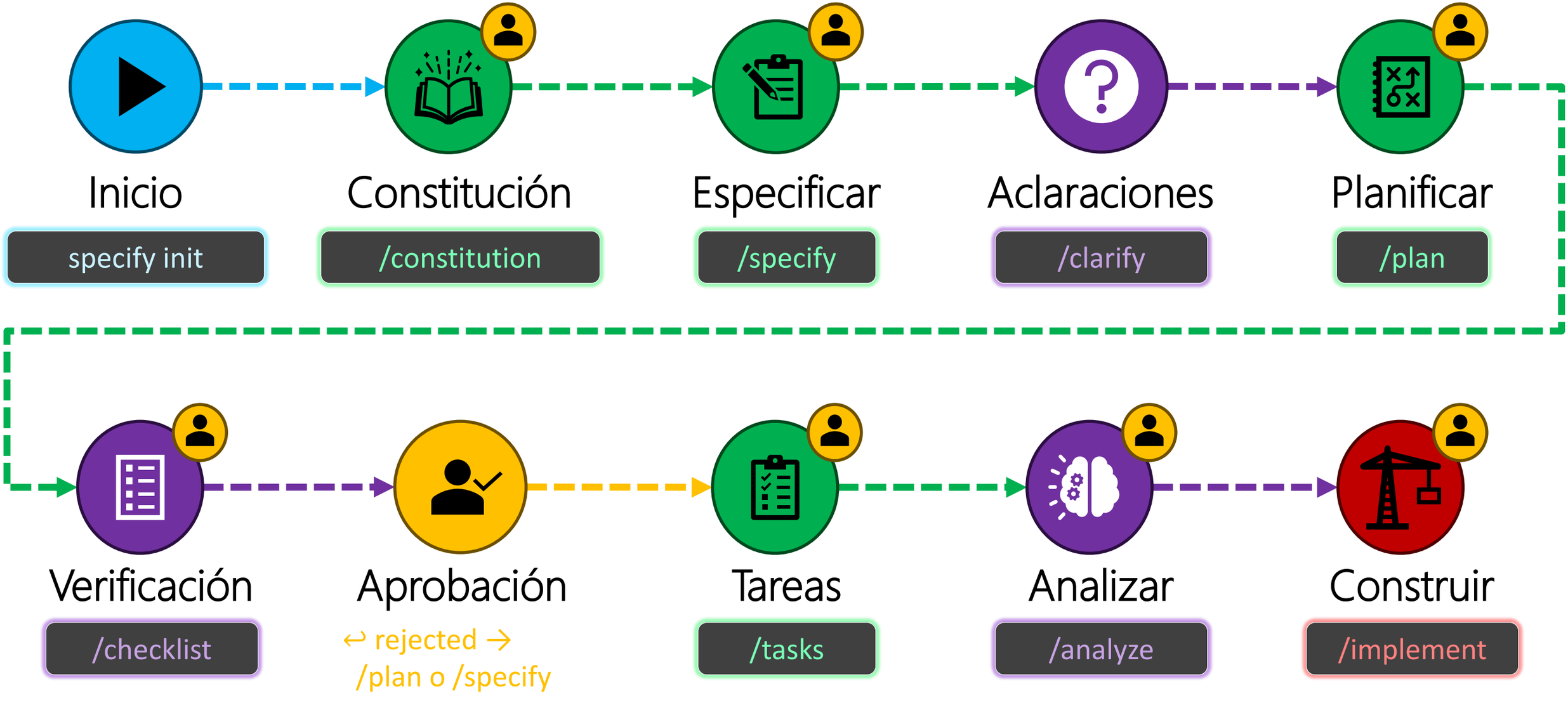
Comandos Slash /

Comandos opcionales – ENHANCED
Altamente recomendados

COMANDO	FUNCIÓN O PROPÓSITO
`/speckit.clarify`	Permite a la IA realizar preguntas estructuradas (aclaraciones) a la persona humana para reducir riesgos en áreas ambiguas (justo antes del `plan`).
`/speckit.checklist`	Genera listas de verificación de calidad para validar la completitud, claridad y consistencia de los requisitos (justo después del `plan`).
`/speckit.analyze`	Análisis de consistencia (*cross-reference*) entre artefactos (después de `tasks` y justo antes `implement`).
`/speckit.converge`	Evalúa el código frente a la especificación, el plan y las tareas para asegurar que nada falte. No modifica el código. Añade tareas de funcionalidades faltantes.

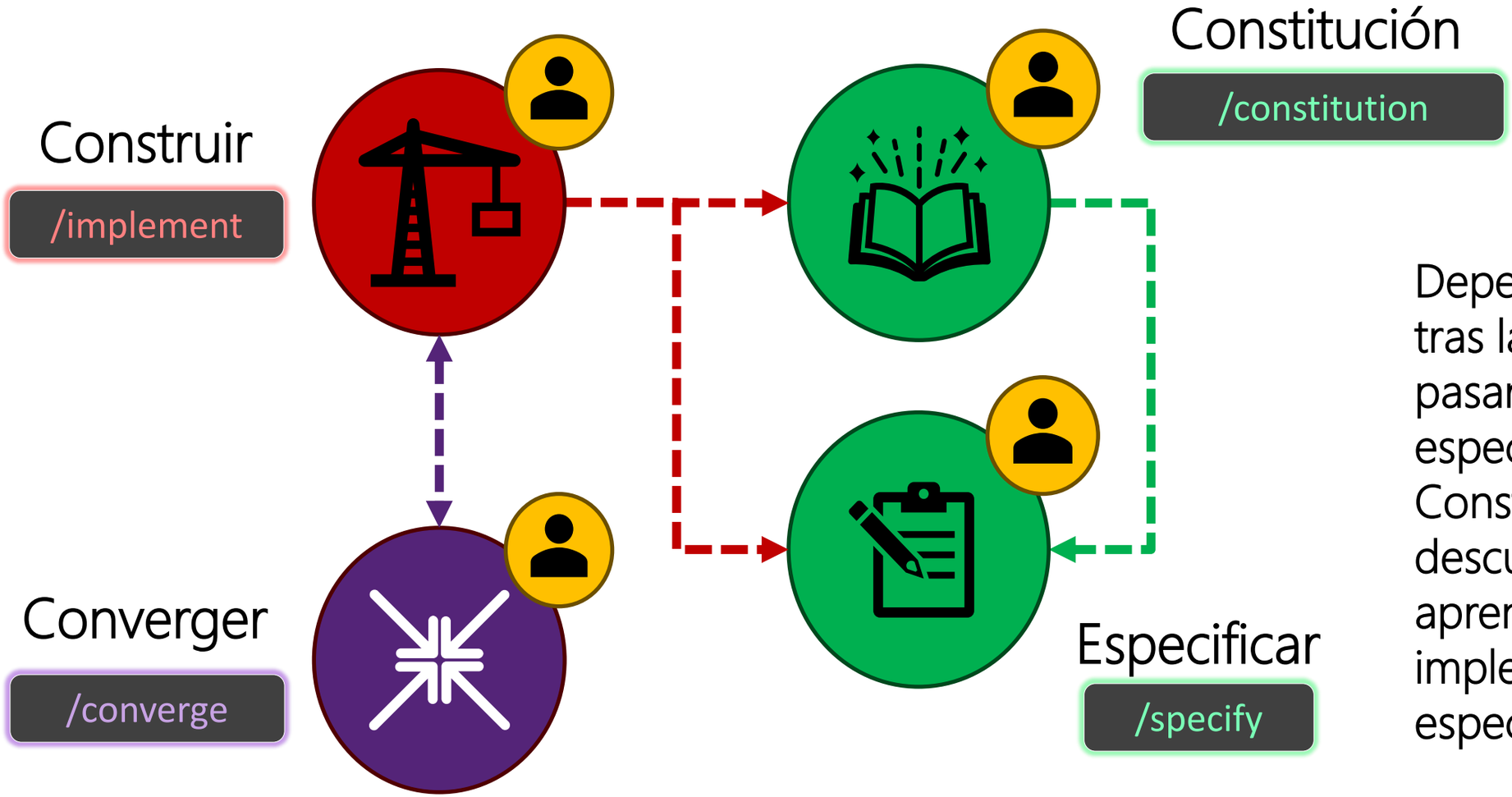
Es muy recomendable usar un modelo de IA diferente del empleado para crear las especificaciones. Así mismo se recomienda modelos razonantes.

GitHub Spec Kit – Flujo de Trabajo



Comando CLI · Comando Core · Comando Enhanced · Actuación humana (*Human-in-the-Loop*) · Actuación de la IA

GitHub Spec Kit – Flujo de Trabajo



Dependiendo de cada caso, tras la construcción se puede pasar a una nueva especificación, o emendar la Constitución con detalles descubiertos y lecciones aprendidas durante la implementación de la especificación anterior.

Comando Core · Comando Enhanced · Actuación humana (*Human-in-the-Loop*) · Actuación de la IA

encamina

Extensiones y Presets

PERSONALIZAR SPEC KIT SIN CAMBIAR EL CORE

EXTENSIONES

Añaden comandos, *hooks* y *scripts* al flujo principal. Ejemplo: automatizar ramas con Git o Azure DevOps.

`.specify/extensions/<id>/`

PRESETS

Sustituyen o componen plantillas y comandos. Ejemplo: un flujo más ligero basado en Lean.

`.specify/presets/<id>/`

Pueden convivir: una extensión gobierna operaciones; un *preset* ajusta artefactos e instrucciones de IA. Ambos permiten personalizar GitHub Spec Kit sin afectar el core de la herramienta entre actualizaciones.

GitHub Spec Kit

EXTENSIONS

Permiten expandir GitHub Spec Kit con funcionalidades extra o adicionales. Es ideal para cuando se necesita ir más allá de lo que ofrece la herramienta por defecto.

Las extensiones introducen nuevos comandos y nuevas plantillas, por ejemplo, añadiendo flujos de trabajo específicos de un dominio que no están cubiertos por los comandos proporcionados, integrándose con herramientas externas o incorporando fases de desarrollo totalmente nuevas.

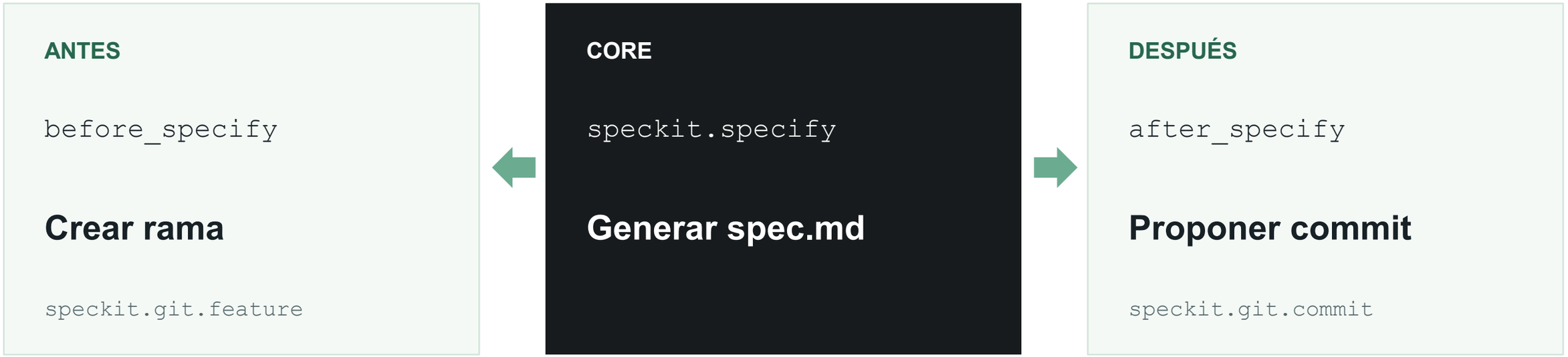
Básicamente es agregar pasos al flujo de trabajo de GitHub Spec Kit para cubrir áreas procedimentales, regulatorias o conductuales propias de la forma de trabajar de la organización, sin afectar a los comandos o pasos actualmente definidos, permitiendo que se realicen modificaciones al flujo de trabajo más allá del recetario de la propia herramienta.

Así, si una organización trabaja con Azure DevOps, podría encontrar interesante evaluar usar la extensión de comunidad que se ha creado para ello → https://github.com/pragya247/Spec Kit-azure-devops

Extension Hooks

ANTES Y DESPUÉS DE UN COMANDO

Un *hook* asocia un evento de GitHub Spec Kit con un comando de una extensión para que se ejecute antes o después de forma transparente, y la mayoría de las veces, también automáticamente. Por ejemplo, la extensión Git de GitHub Spec Kit incluye, entre otros, el siguiente *hook*:



Son *hooks* del flujo GitHub Spec Kit, no son *hooks* nativos de Git ni de GitHub Copilot.

GitHub Spec Kit

PRESETS

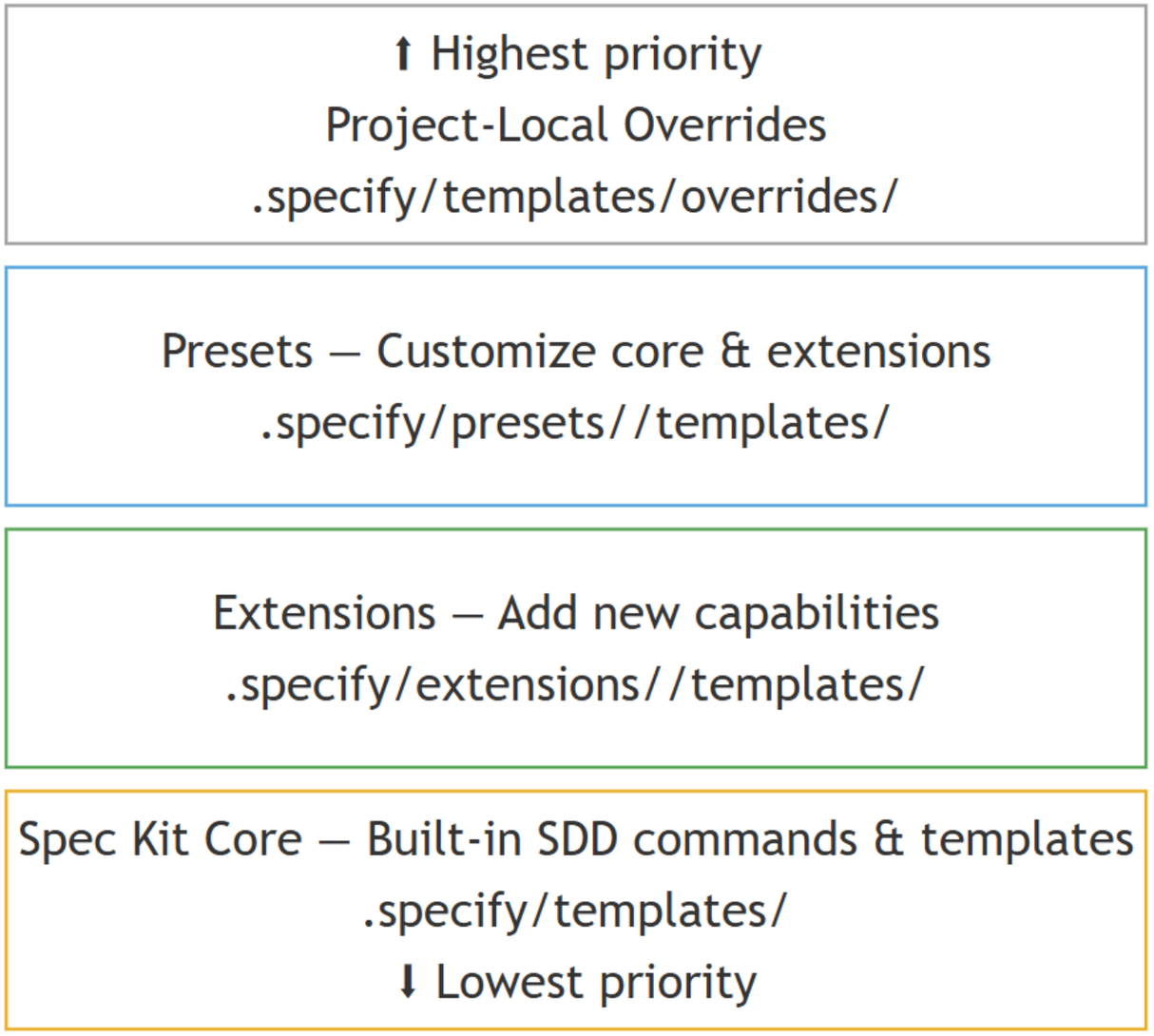
Los *presets* sobrescriben las plantillas y los comandos que vienen por defecto en GitHub Spec Kit, y se utilizan cuando se necesita modificar como opera la herramienta sin intervenir directamente en los artefactos que proporciona la misma por defecto.

Por ejemplo, los *presets* pueden reestructurar las plantillas de especificación (los Markdown) para exigir trazabilidad regulatoria, adaptar el flujo de trabajo a la metodología que utilice la organización (Agile, Kanban, Waterfall, etc.), añadir pasos obligatorios de revisión de seguridad en los planes, imponer un orden de tareas orientado a *test-first* o incluso localizar todo el flujo de trabajo a otro idioma diferente del inglés.

Se pueden apilar varios *presets* con un orden de prioridad.

Hay un ejemplo muy gracioso para hacer que GitHub Spec Kit actúe como un pirata →
https://github.com/mnriem/Spec Kit-pirate-speak-preset-demo

Presets: una pila de personalización



Los *presets* se utilizan en base a una pila o prioridad descendente de determinación.

Así, los elementos en la carpeta **`overrides`**, que corresponden a personalizaciones *ad-hoc* del proyecto, tienen más prioridad que los *presets* definidos en la carpeta **`presets`**, los cuales a su vez tienen más prioridad que cualquier *preset* que agregue una extensión.

Finalmente, como *fallback* o último recurso, tenemos las plantillas del propio GitHub Spec Kit.

GitHub Spec Kit

PRESETS

Plantillas y Comandos

Las plantillas se resuelven en tiempo de ejecución. GitHub Spec Kit recorre la pila de *presets* de arriba hacia abajo y utiliza la primera coincidencia.

Las sobrecargas locales del proyecto permiten hacer ajustes puntuales para un único proyecto sin necesidad de crear una configuración (*preset*) completa. Se colocan en `.specify/templates/overrides/`

Los comandos se aplican en el momento de la instalación: cuando se ejecuta `specify extension add` o `specify preset add`, los archivos de comandos se escriben en los directorios de los agentes.

Si varias configuraciones (*presets*) o extensiones proporcionan el mismo comando, prevalece la versión con mayor prioridad. Al eliminar uno, se restaura automáticamente la siguiente versión con mayor prioridad.

Si no existen sobrecargas ni personalizaciones, GitHub Spec Kit utiliza sus valores predeterminados del núcleo.

GitHub Spec Kit

PRESETS & EXTENSIONS

```
# Search available extensions
specify extension search

# Install an extension
specify extension add <extension-name>
```

```
# Search available presets
specify preset search

# Install a preset
specify preset add <preset-name>
```

encamina

GitHub Spec Kit – La importancia de la Constitución

La Constitución es, seguida de las especificaciones, la documentación donde más se debe aplicar conocimiento, trabajo y esfuerzo.

Los beneficios se vuelven poderosos en la práctica gracias a la aplicación automática. Si la Constitución establece "*todos los datos deben estar cifrados en reposo*", el agente de IA no sugerirá almacenar archivos en texto plano: incorpora automáticamente el cifrado en los diseños.

GitHub Spec Kit – La importancia de la Constitución

Una Constitución bien estructurada organiza los principios en categorías (habitualmente llamadas capítulos) que la IA pueden consultar fácilmente.

Las secciones más comunes incluyen:

- Estándares tecnológicos.
- Requisitos de seguridad.
- Rendimiento y escalabilidad.
- Estándares de codificación.
- Cumplimiento y gobernanza.

GitHub Spec Kit – La importancia de la Constitución

```
## Technology Standards

- All cloud resources must be hosted on Microsoft Azure.
- Back-end services use .NET 8 or later.
- Front-end applications use React or Blazor.
- Database: Azure SQL Database or Cosmos DB (no on-premises SQL Server).
- Secret management exclusively via Azure Key Vault.
```

```
## Security Requirements

- Authenticate all API requests using Microsoft Entra ID tokens.
- Encrypt all data at rest using AES-256 encryption.
- Encrypt data in transit using TLS 1.2 or higher.
- Never log personally identifiable information (PII).
- Store no secrets in source code or configuration files.
- Implement role-based access control (RBAC) for all features.
```

GitHub Spec Kit – La importancia de la Constitución

```
## Performance and Scalability

- APIs must respond within 200ms for 95th percentile requests.
- System must handle 10,000 concurrent users.
- Use asynchronous processing for operations exceeding 5 seconds.
- Implement caching for frequently accessed data.
- Design for horizontal scalability using Azure App Service or Container Apps.
```

```
## Coding Standards

- Follow Microsoft C# Coding Conventions.
- Maintain minimum 80% unit test coverage.
- All public APIs documented with XML comments.
- Use dependency injection for service dependencies.
- Implement structured logging using ILogger interface.
```

GitHub Spec Kit – La importancia de la Constitución

```
## Compliance and Governance

- Comply with local and international standards that protect personal information for all user data processing.
- Implement audit logging for all data modifications.
- Support accessibility standards (WCAG 2.1 Level AA).
- Enable monitoring and alerting via Azure Application Insights.
- Retain logs for minimum 90 days for compliance audits.
```

Al agregar GitHub Spec Kit a un proyecto existente que aún no tiene una Constitución, se puede generar una utilizando el comando «`/speckit.constitution`», pero indicando de forma explícita que se revise con detenimiento la actual base de código existente para tratar de inferir los principios y restricciones del proyecto al crear un archivo inicial. Siempre es recomendable revisar y refinar la constitución generada para asegurar que refleje con precisión los requisitos del proyecto.

GitHub Spec Kit – La importancia de la Constitución

Para refinar, considerar los siguientes pasos:

Agregar principios faltantes: Si se omiten requisitos importantes, es muy recomendable añadir el contenido manualmente. Por ejemplo, si la organización exige formatos específicos de registro (*logging*), incorpora esa casuística.

Eliminar contenido innecesario: Se podría generar texto genérico o principios demasiado amplios. Es conveniente revisar si el contenido es valioso, y en caso contrario es mejor eliminar todo lo que no aporte una guía concreta.

Hacer que los principios sean específicos y comprobables: Sustituir afirmaciones vagas por criterios medibles. Por ejemplo, cambiar "*el sistema debe ser rápido*" por "*las respuestas de la API deben completarse en máximo 200 ms para el 95% de las solicitudes*".

Alinear con los estándares internos: Hacer referencias concretas a documentos existentes dentro del repositorio cuando sea posible. Por ejemplo: "*seguir las directrices de Engineering Excellence v2.3 para el diseño de APIs*". Incluir el enlace.

Verificar con las partes interesadas: Compartir la constitución con *stakeholders*, equipos de seguridad, cumplimiento (legal) y arquitectura para confirmar que recoge todos los requisitos necesarios.

GitHub Spec Kit – La importancia de la Constitución

Al crear constituciones para proyectos internos de categoría empresarial, es extremadamente recomendable incorporar los siguientes requisitos comunes:

Arquitectura "*-first*": Exigir el uso de servicios (por ejemplo, Azure, AWS) para alojamiento, almacenamiento, bases de datos e infraestructura de soporte. Este requisito se alinea con la estrategia corporativa de nube y las herramientas disponibles.

Identidad empresarial: Especificar la tecnología (por ejemplo, Microsoft Entra ID) para la autenticación y la autorización. La mayoría de las aplicaciones internas empresariales se integran con los sistemas de identidad corporativos.

Seguridad y cumplimiento: Hacer referencia a las políticas de seguridad de la empresa, los principios y los marcos de cumplimiento aplicables.

Supervisión y observabilidad: Exigir el uso de soluciones de monitoreo. Las aplicaciones empresariales necesitan telemetría completa para el soporte en producción.

Estándares de herramientas: Especificar los marcos de trabajo (*frameworks*), librerías y prácticas de desarrollo aprobados. Por ejemplo, exigir el uso de paquetes NuGet gestionados desde repositorios internos o aprobados por seguridad. Uso de MCP (*Model Context Protocol*), tanto institucionalmente como localizado a nivel de proyecto.

Accesibilidad: Incluir los requisitos de cumplimiento con WCAG 2.1 Nivel AA. Por ejemplo, para los escenarios donde las herramientas o servicios puedan ser accesibles para personas con discapacidades.

GitHub Spec Kit – La importancia de las especificaciones

Durante una iteración, la especificación (**spec.md**) es la única fuente de verdad sobre lo que debe hacer el software. Escribir especificaciones claras, descriptivas, concretas e inequívocas es clave.

No es algo fácil, requiere de muchísima práctica, implicación, recurrencia, diligencia y disciplina.

En ese sentido, prácticas como «*Las Siete C's de la Comunicación*» son altamente recomendables.

Ser específico y medible: Reemplazar términos imprecisos por valores precisos. Por ejemplo, en vez de "*admite archivos grandes*" especificar "*admite archivos de hasta 50 MB*", o en vez de "*rendimiento rápido de carga*" indicar explícitamente que "*las cargas de menos de 5 MB se completan en un plazo de 5 segundos en la conexión de 10 Mbps*". Los **requisitos específicos permiten a la IA generar implementaciones** que satisfagan sus necesidades **reales**, incluyendo pruebas de integración y rendimiento apropiadas.

Usar terminología coherente: Definir los términos una vez y reutilizar a lo largo de la especificación. Si se usa "*documentos*", no cambiar por "*archivos*" ni "*datos adjuntos*" más adelante. **La terminología incoherente o un léxico plagado de sinónimos confunde** tanto a los seres humanos como **a la IA**. En el caso de los proyectos internos de la empresa, usar los nombres oficiales de productos y la terminología de los estándares organizacionales. Puede valer la pena incluir un archivo `*.instructions.md` de glosario y definiciones.

GitHub Spec Kit – 7 *C's de la Comunicación*



Cuando se **gane una oportunidad**, el sistema deberá enviar los datos a Business Central **rápidamente** y gestiona los **posibles errores**.

ERROR: ¿Qué significa Exactamente que esté ganada? ¿Qué datos se envían?

ERROR: Ambigüedad métrica ¿segundos? ¿Minutos)

ERROR: Excepción no Controlada. ¿Cuántos reintentos? ¿Cómo evitamos duplicados?

C1 Clara

El sistema debe gestionar correctamente las oportunidades

El sistema debe iniciar la sincronización al cambiar al estado Ganada.

(Identifica el evento exacto).

C2 Concisa

Registrar y guardar información de errores para que los desarrolladores puedan consultar posteriormente…

Registrar cada error con fecha, código y mensaje.

(Elimina redundancia).

C3 Concreta

La sincronización debe ser rápida.

El 95% de las solicitudes debe procesarse en < 2 segundos.

(Métrica verificable mediante pruebas)

C4 Correcta

Refleja la realidad del negocio y restricciones técnicas.
(ej, La regla real exige que la oportunidad tenga un contrato asociado, no solo el estado 'Ganada').

C5 Coherente

Cero contradicciones inter-documentos. Sincronización perfecta entre constitution.md, spec.md, plan.md y tasks.md.
(Ej. Si spec pide 3 reintentos, el plan técnico no puede proporner 5).

C6 Completa

Cobertura total de los caminos felices y los casos límite (Edge Cases).
(ej. Fallos de red, Business central caído, límites de reintentos, prevención de duplicados).

C7 Contextual

Sea legible y orientada al lector, utilizar títulos descriptivos consistentes, definir acrónimos de negocio técnicos, separación estricta entre requisitos funcionales y restricciones técnicas, evitar párrafos excesivamente largos. Respetar el tiempo del revisor y la ventana de contexto de la IA

encamina

GitHub Spec Kit – La importancia de las especificaciones

Abordar el manejo de errores explícitamente: Suponer que la IA controla correctamente los errores es un error. Es fundamental especificar qué ocurre y cómo ha de comportarse o responder el software cuando se produce un error en las operaciones.

Mantener el ámbito adecuado: Si una característica requiere más de 300 líneas para especificar, considerar mejor la posibilidad de dividirla en varias especificaciones. Por ejemplo, en vez de una única y megalítica especificación de "*Sistema de administración de documentos*" puede ser más rentable crear especificaciones independientes y relacionadas a acciones tales como "*Carga de documentos*", "*Descarga de documentos*", "*Uso compartido de documentos*" y "*Búsqueda de documentos*". Las **especificaciones basadas en características, con descripciones más concretas son más fáciles de revisar**, aclarar **e implementar**, y también se alinean con las prácticas de entrega incremental.

Detallar el "qué", no el "cómo": Quizás sea **la recomendación más fundamental, pero con sutilezas.** Y es que las **especificaciones definen requisitos, no implementaciones**. Indicar lo que debe hacer el sistema, no cómo codificarlo. Por ejemplo, en vez de "*usar el paquete NuGet Azure.Storage.Blobs con la clase BlobContainerClient*" indicar "*almacenar archivos cargados en Azure Blob Storage*". Las decisiones de implementación pertenecen a la fase del plan. Sin embargo, hay matices, y si la Constitución exige tecnologías específicas o existe una necesidad inminente, es válido hacer referencia a detalles de implementación en la especificación.

GitHub Spec Kit – La importancia de las especificaciones

Abordar el manejo de errores explícitamente: Suponer que la IA controla correctamente los errores es un error. Es fundamental especificar qué ocurre y cómo ha de comportarse o responder el software cuando se produce un error en las operaciones.

Mantener el ámbito adecuado: Si una característica requiere más de 300 líneas para especificar, considerar mejor la posibilidad de dividirla en varias especificaciones. Por ejemplo, en vez de una única y megalítica especificación de "*Sistema de administración de documentos*" puede ser más rentable crear especificaciones independientes y relacionadas a acciones tales como "*Carga de documentos*", "*Descarga de documentos*", "*Uso compartido de documentos*" y "*Búsqueda de documentos*". Las **especificaciones basadas en características, con descripciones más concretas son más fáciles de revisar**, aclarar **e implementar**, y también se alinean con las prácticas de entrega incremental.

Detallar el "qué", no el "cómo": Quizás sea **la recomendación más fundamental, pero con sutilezas.** Y es que las **especificaciones definen requisitos, no implementaciones**. Indicar lo que debe hacer el sistema, no cómo codificarlo. Por ejemplo, en vez de "*usar el paquete NuGet Azure.Storage.Blobs con la clase BlobContainerClient*" indicar "*almacenar archivos cargados en Azure Blob Storage*". Las decisiones de implementación pertenecen a la fase del plan. Sin embargo, hay matices, y si la Constitución exige tecnologías específicas o existe una necesidad inminente, es válido hacer referencia a detalles de implementación en la especificación.

GitHub Spec Kit + SDD

Aunque SDD destaca en proyectos nuevos (*greenfield*), los entornos empresariales suelen trabajar con bases de código existentes (*brownfield*).

Al añadir Spec Kit a proyectos ya establecidos, primero debemos considerar:

- Se deben realizar acciones de ingeniería inversa cuyo resultado debe ser artefactos y productos documentales que describan con máximo detalle las características funcionales y técnicas del proyecto. Esto se suele denominar “code elevation”.
- La Constitución debe documentar los patrones arquitectónicos y las restricciones o limitaciones existentes, así como la pila tecnológica (*technology stack*) actual.
- Las Especificaciones deben reconocer la infraestructura actual, los puntos de integración y cada aspecto funcional.
- Los planes han de mostrar cómo las nuevas funcionalidades se integran con la arquitectura actual en lugar de proponer implementaciones aisladas o incluso refactorizaciones (a menos que se solicite explícitamente).

Progressive Disclosure

Es el principio de revelar solo el contexto necesario para la tarea actual. Es una dinámica empleada en muchas disciplinas, y para SDD, implica organizar el conocimiento de forma ordenada y gobernada.

Es buscar proporcionar información especializada bajo demanda y reducir el ruido contextual.

En el mundo de la IA, esto se refiere a indicar al agente exclusivamente la información precisa, concisa y específica que necesita para realizar una tarea en un momento dado.

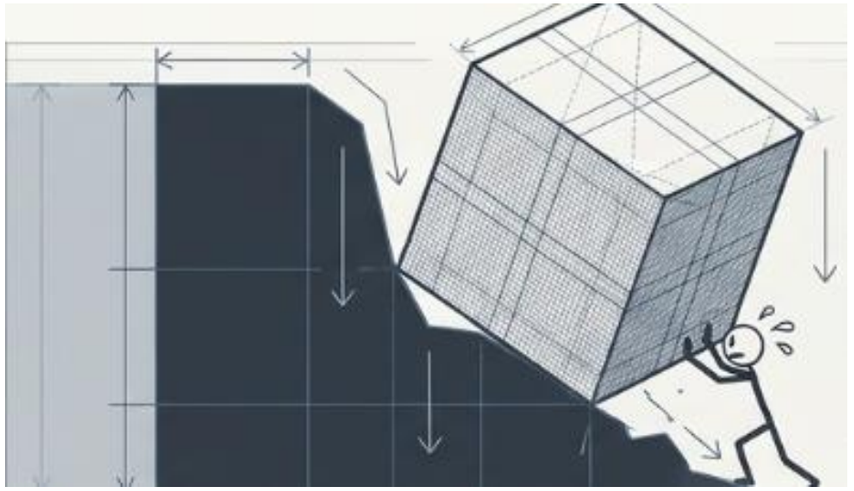
El beneficio inmediato es un ahorro en tokens (*context window*) y minimización relativa de alucinaciones o ruido al momento de actuar.

Se suele traducir como Divulgación Progresiva.

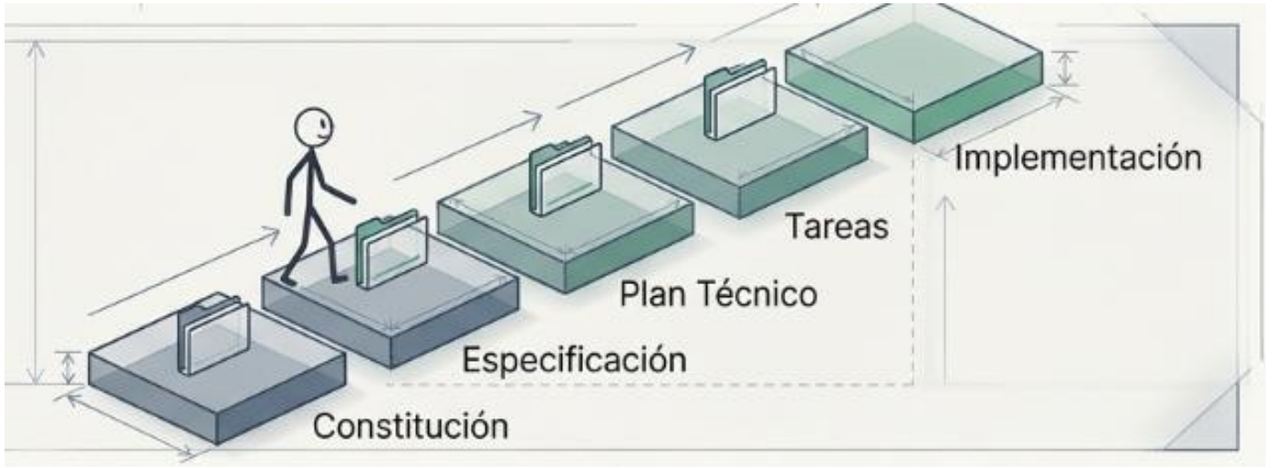


El Cambio de Paradigma

Enfoque Big Ban:
Alto riesgo de fallo en cadena



Progresive Disclosure:
Trabajo acotado y gradual

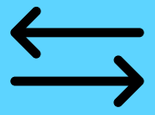


Los 6 Principios del Progressive Disclosure



Minimalismo

Definir solo reglas necesarias para todas las tareas del repositorio.



Separación

Instrucciones especializada van en archivos específicos. Nada de "*documento único*" con todo adentro.



Delegación

Declarar donde localizar instrucciones especializadas mediante enlaces. El agente las busca cuando hace falta.



Accionabilidad

Preferir instrucciones directas. Evitar textos genéricos como "*sigue buenas practicas*".



Relevancia

Revelar solo la información necesaria para la tarea o necesidad actual. Evitar sobrecargadas con datos innecesarios y mantener el enfoque en lo que realmente importa.



Incremental ~ Progresivo

Una vez que conceptos y acciones estén claras o ya implementadas, introducir nuevos requerimientos, especificaciones o cambios.

encamina

PROGRESIVE DISCLOSURE

Comienza con una **visión de alto nivel** y deja que la **IA redacte los detalles**

- Comienza con **una especificación de alto nivel**: objetivo, requisitos esenciales y propósito.
- Deja **que la IA expanda** ese resumen (*brief*) en una especificación detallada con funcionalidades, arquitectura, modelo de datos, plan paso a paso.
- **Revisa, corrige o modifica la expansión realizada por la IA**, la cual es la fuente de verdad compartida entre tu y la IA, pero recordando que **tu serás el único y exclusivo responsable**, por lo que **es importante asegurar alineación** y evitar desviaciones o alucinaciones.
- En las especificaciones, **mantén el enfoque en el «qué»** y a veces en el «por qué», **no en el «cómo»** técnico, al menos inicialmente o si no es inminentemente necesario.
- **Guarda la especificación** (p. ej., `SPEC.md`) y opcionalmente úsala como contexto persistente en sesiones futuras. La especificación actúa como un PRD (*Product Requirements Document*): mantiene el proyecto alineado y mejora la calidad del código generado.

PROGRESIVE DISCLOSURE

Estructura y contenido esencial

- Nuevamente, **trata las especificaciones como un PRD**. Deben ser documentos estructurado con secciones claras, no notas sueltas
- Atienda a las seis áreas clave: comandos, pruebas (*testing*), estructura del proyecto, estilos de codificación, flujos de protección (uso de Git) y límites o restricciones (cosas a nunca tocar).
- Cuando corresponda y sea fundamentalmente necesario, sé específico con la pila tecnológica (*technology stack*). Incluye versiones y dependencias (p. ej., React 18 + TypeScript, Vite, Tailwind, etc.).
- **Formato consistente**: usa Markdown con encabezados (*frontmatter*) para que la IA parsee fácilmente la información. **Elige un lenguaje de comunicación** (inglés, español, …).
  - Desde **ENCAMINA** recomendamos el inglés.
- Para todo esto, **apóyate en archivos de instrucciones o habilidades (*skills*) para contextualizar a la IA** en cómo escribir este tipo de documentos.

PROGRESIVE DISCLOSURE

Flujo de trabajo y gobernanza

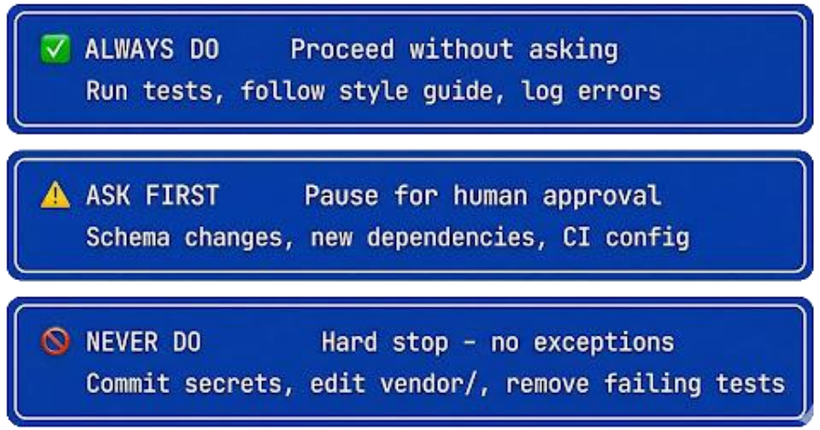
- Usa siempre el flujo en 4 fases:
  Specify → Plan → Tasks → Implement
- **Valida y revisa en cada fase** antes de avanzar con la siguiente.
- Evalúa utilizar dentro del flujo de trabajo las fases opcionales:
  Clarify → Checklist → Analyze
  - Desde **ENCAMINA** recomendamos siempre usar estas fases, no considerarlas como opcionales, sino como obligatorias.
- **Los documentos están vivos**: versiona y actualiza las especificaciones, actualiza y versiona la constitución, etc.
- **Cada cierto tiempo actualiza el Spec Kit**:
  - Reinstala para actualizar el CLI:
    uv tool install specify-cli --from git+https://github.com/github/Spec Kit.git
  - Reinicializa para para actualizar las plantillas (*templates*):
    specify init --here --force

encamina

PROGRESIVE DISCLOSURE

Flujo de trabajo y gobernanza

- Es recomendable **establecer** al menos tres (3) **niveles de actuación**:
  - Nivel 1 - Siempre: son acciones automáticas o necesarias.
  - Nivel 2 - Aclarar: identificación de cambios de alto impacto o establecimiento de niveles de confiabilidad antes de actuar.
  - Nivel 3 - Nunca: prohibiciones absolutas. Situaciones cuya ocurrencia debe minimizarse al máximo posible.



- **Juicios a través de LLMs**. Utiliza otros agentes para evaluaciones subjetivas (estilo, arquitectura, legibilidad) que emplee un modelo de IA diferente a los utilizados en la generación de los artefactos a evaluar.

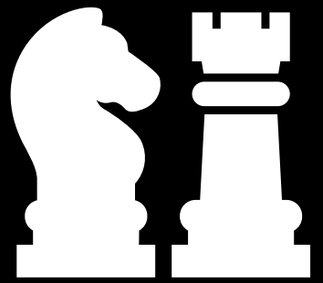
PROGRESIVE DISCLOSURE

Herramientas, paralelización y costes

- **Gestión del contexto**: usa patrones como RAG (*Retrieval-Augmented Generation*) o MCP (*Model Context Protocol*) como herramientas para recuperar secciones relevantes para cada fase del flujo de trabajo.
- **No uses todas las herramientas a la vez**. Con cada interacción, elije el conjunto más apropiado y más reducido (no llenes cada petición de MCPs)
- Herramientas propias de un proyecto, deben ser gestionadas a nivel del proyecto, y no del espacio de trabajo. En el espacio de trabajo usa herramientas que requiera autenticación mediante claves o certificados.
- Optimiza costes y velocidad. **No te quedes o utilices un único modelo de IA para todo**. Cada modelo tiene fortalezas y debilidades, y cambia su comportamiento de persona en persona.
  - Desde **ENCAMINA** recomendamos modelos ligeros para borradores y documentación, y modelos potentes para pasos críticos, codificaciones avanzadas o análisis complejos.



DEMO



encamina

¡Gracias! Para localizar o contactar con ENCAMINA puedes:

Enviar un mail a:

info@encamina.com
administracion@encamina.com

O llamar al:

Madrid +34 917 893 823
Valencia +34 962 698 064

O enviar un fax al 962 698 063

O hablar personalmente con tu Account Manager

O visitarnos en:

C/O'Donnell, 34
28009, **Madrid**

C/Fundició, 79
46520 Port de Sagunt, **València**

InnovaParq Universidad de La Laguna
Av. Trinidad, 61. Campus Central
38200, S. Cristóbal de La Laguna, **Tenerife**

Puedes encontrarnos en:

